Literature Review - Temporal CNN for DDoS Attack Detection

Lokesh L K S (ljl5837@psu.edu)

The Pennsylvania State University

1st February 2026

1. Literature Review

1.1 Introduction

This literature review looks at current AI and cybersecurity research that's relevant to my project on temporal CNN-based DDoS detection. I focused on finding sources about production deployment strategies, deep learning architectures, and practical constraints for implementing AI-driven security solutions. My sources include peer-reviewed papers, Cloudflare's technical documentation, and comparative studies on the BCCC-cPacket-Cloud-DDoS-2024 dataset that I'm using for this project.

1.2 Production DDoS Detection: Cloudflare's Approach

Cloudflare's DDoS protection system shows how real-world detection actually works in production. They use three layers: Edge/Network Layer, Local Detection/Mitigation Layer, and Global Intelligence Layer. What's interesting is their out-of-path traffic analysis approach - they process traffic samples asynchronously so legitimate traffic doesn't get slowed down. They can mitigate attacks in about 3 seconds on average and detect 98.6% of L3/L4 attacks and 81% of L7 attacks. Their Global Intelligence Layer builds dynamic baselines using seven-day rolling windows, which shows that temporal pattern recognition is pretty essential for telling apart attacks from legitimate traffic spikes. This gives some good benchmarks to strive for: detection time under 3 seconds, out-of-path processing capability, and temporal pattern modeling.

1.3 DDoSBERT: Transformer-Based Detection

Le et al. (2025) published research on DDoSBERT in Computer Networks, which takes a different approach by converting numerical network features into text tokens and using transformer architectures for detection. They tested it on the BCCC-cPacket-Cloud-DDoS-2024 dataset, and got accuracy ranging from 93.97% to 98.98% depending on which feature selection method they used - Mutual Information gave them the best results at 98.98%. They trained using AdamW optimizer with learning rate 1e-5, batch size 8, and 5 epochs. The problem is DDoSBERT needs a lot of computational resources because of the transformer complexity and the text tokenization preprocessing step. So there's this trade-off: It can get really high accuracy (over 98%) but it needs resource-intensive architectures that wouldn't work well for edge deployment.

1.4 Random Forest vs. Neural Networks Study

Wolosewicz et al.'s comparative study on the BCCC dataset revealed some important insights about handling class imbalance. They used Random Forest with recursive feature elimination to reduce 315 features down to just 6 temporal attributes (things like packet inter-arrival time max, forward packets IAT max), and achieved 97% accuracy. For the neural network part, they tried a deep architecture with 10 Dense layers and 50-neuron width, which got 96.3% accuracy. But here's the catch - it only correctly classified 2 out of 5 held-out attacks. Turns out the model was just exploiting the class distribution (way more benign flows than attacks) instead of actually learning attack patterns. When they retrained with a balanced 50-50 validation set, attack detection improved to 4 out of 5 correct, but overall accuracy dropped

to 90%. This shows that neither aggressive feature reduction nor deep architectures with standard features really capture temporal attack evolution patterns. That's why converting sequential flows into temporal images might be necessary.

1.5 LUCID: Lightweight CNN Architecture

Doriguzzi-Corin et al. (2020) published LUCID in IEEE Transactions on Network and Service Management as a lightweight CNN specifically designed for edge devices with limited resources. Their preprocessing groups bi-directional flow packets within time windows (1-100 seconds) and extracts 11 packet-level attributes. The CNN architecture is pretty simple - just one convolutional layer with 64 filters, max pooling, and a fully-connected layer - only 2,241 trainable parameters total. Despite being lightweight, it achieved F1 scores between 0.9888 and 0.9987 on different datasets while processing over 55,000 samples per second, which is 40x faster than competing approaches. They validated it on an NVIDIA Jetson TX2 board and showed it could process 1.9 million packets per second and work within 1GB RAM constraints. The main limitation is that LUCID treats each time window independently - it doesn't model temporal evolution across multiple windows. That's what temporal CNN approach aims to fix by converting sequential flow segments into temporal images so the CNN can learn both spatial patterns within windows and temporal attack progression across consecutive windows.

2. Requirements Definition

2.1 Functional Requirements

- **FR-1: Dataset Processing**: The system needs to process sequential flows from the BCCC-cPacket-Cloud-DDoS-2024 dataset, group them into time windows, and convert them to 2D temporal images normalized to [0-255] pixel values.
- **FR-2: Temporal Pattern Learning**: The system should capture both spatial feature patterns within individual time windows and temporal attack progression patterns across consecutive windows.
- **FR-3: Class Imbalance Handling**: The system needs to implement SMOTE or something equivalent for balancing to avoid the model just exploiting class distribution instead of learning actual attack patterns.
- **FR-4: Binary Classification**: The system should do binary classification (Attack/Benign) with probability output, classifying samples as DDoS when $p > 0.5$.

2.2 Non-Functional Requirements

- **NFR-1: Detection Accuracy**: I'm targeting at least 80% accuracy on the BCCC test set as the baseline threshold for acceptable performance.
- **NFR-2: False Negative Minimization**: Prioritize minimizing missed attacks over false positives since missing an actual attack is worse for security.
- **NFR-3: Resource Efficiency**: Achieve decent accuracy with way lower computational overhead than transformer-based approaches like DDoSBERT.

2.3 Success Criteria

My project will be considered successful if I can achieve the following:

1. Get at least 80% classification accuracy on the BCCC-cPacket-Cloud-DDoS-2024 test set. This is my minimum acceptable threshold.
2. Achieve faster inference time and lower memory footprint compared to DDoSBERT while still maintaining comparable detection capabilities.
3. Correctly classify the majority of held-out attack examples to demonstrate the model is actually learning attack patterns and not just exploiting class distribution.

4. Validate that the temporal CNN approach performs well with minimal resources, making it practical for edge deployment scenarios where high-accuracy transformer models like DDoSBERT would be way too expensive to run.

3. References

Cloudflare. (2024). *A deep dive into Cloudflare's autonomous DDoS protection*. Cloudflare Blog. https://blog.cloudflare.com/deep-dive-cloudflare-autonomous-edge-ddos-protection/

Doriguzzi-Corin, R., Millar, S., Scott-Hayward, S., Martínez-del-Rincon, J., & Siracusa, D. (2020). LUCID: A practical, lightweight deep learning solution for DDoS attack detection. *IEEE Transactions on Network and Service Management*, *17*(2), 876–889. https://doi.org/10.1109/TNSM.2020.2971776

Le, T., Kim, H., Kim, H., & Yoo, M. (2025). DDoSBERT: A novel approach to DDoS attack detection using pre-trained BERT model. *Computer Networks*, *262*, Article 110889. https://doi.org/10.1016/j.comnet.2024.110889

University of New Brunswick, Canadian Institute for Cybersecurity. (2024). *BCCC-cPacket-Cloud-DDoS-2024 dataset*. https://www.unb.ca/cic/datasets/ddos-2024.html

Wolosewicz, M., Damavandinejadmonfared, S., & Erbacher, R. F. (2024). Comparative analysis of machine learning approaches for DDoS attack detection on BCCC-cPacket-Cloud-DDoS-2024 dataset [Conference paper]. *IEEE International Conference on Network and Service Management*. https://www.unb.ca/cic/datasets/ddos-2024.html